第11课 FP8：浮点量化与低精度计算

一组激活中，少数大值可能拉大共享的 INT8 量化步长，使大量小值失去区分度。FP8 使用非等距浮点刻度，在覆盖大值的同时，为小值保留较细的刻度。本课从数值分布与刻度的关系出发，说明 FP8 的缩放、舍入，以及量化后的矩阵计算。

一、FP8 的量化优势与计算收益

1. 大值对共享量化步长的影响

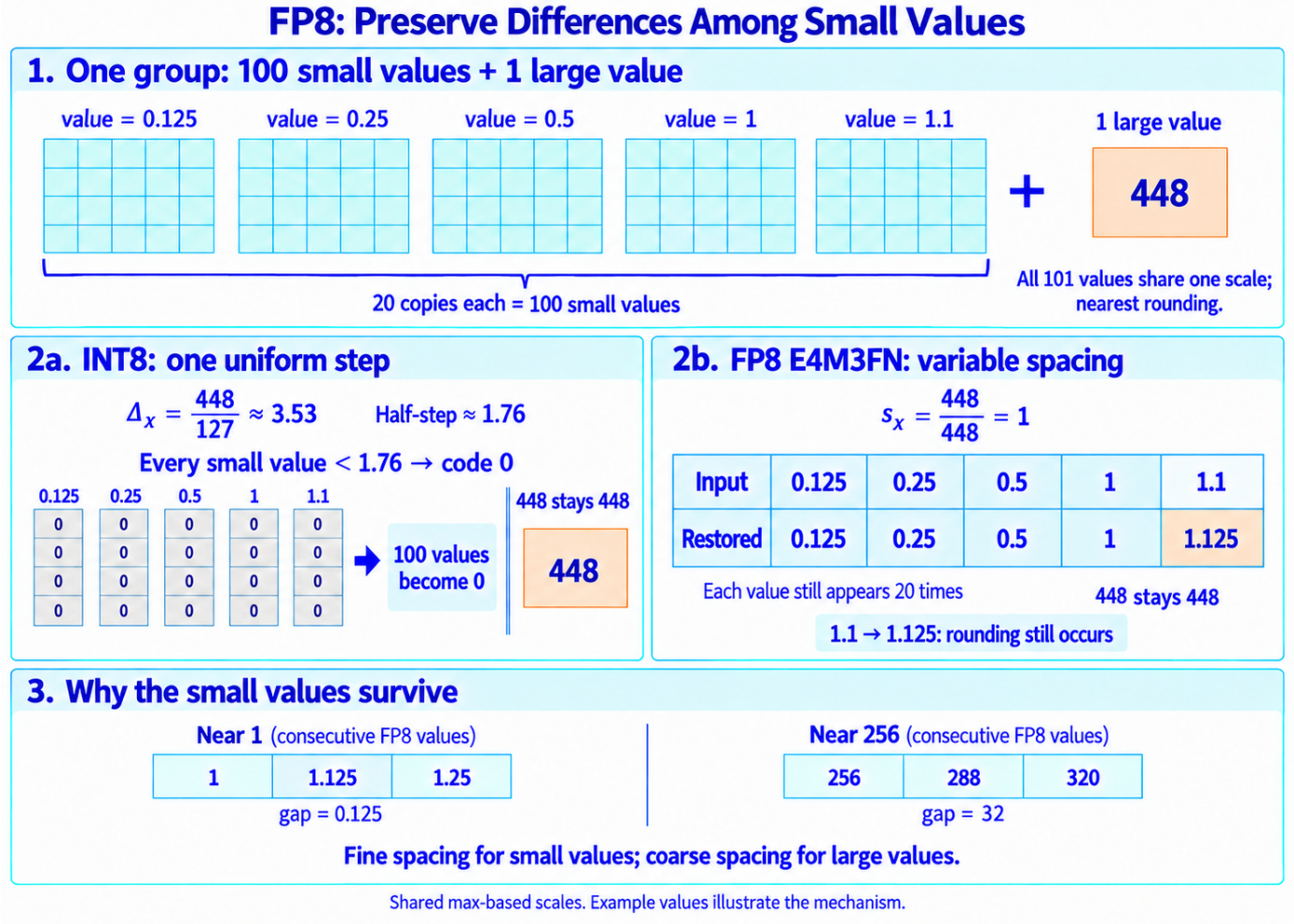
将权重和激活从 16 位降到 8 位，可以减少存储和数据搬运，并利用相应硬件加速计算。但位数减少后，可表示的数值也更少，如何分配这些刻度会直接影响量化误差。

当一组激活中**大量数值较小、少数数值很大** 时，若共用一个按最大绝对值确定的 INT8 步长，大值会拉大整组的刻度间隔。大量小值可能被舍入为零，或被舍入到同一个刻度上，原本的差异随之消失。

2. FP8 的非等距刻度

FP8 保留指数，使刻度随数值大小变化：**小值附近刻度密，大值附近刻度疏。** 因此，在小值集中、大值稀少且跨越多个数量级的数据中，FP8 可能更好地保留小值之间的差异。

图中将 100 个小值与 1 个大值放在同一组内。两种格式分别按这组数据的最大绝对值确定尺度，组内共享尺度，并舍入到最近的可表示值。INT8 使用 −127 到 127 的对称编码范围；FP8 使用最大有限值为 448 的 E4M3FN 格式。INT8 的 Δ_x 是固定量化步长；FP8 的 s_x 是缩放尺度，不代表固定刻度间隔。



两种格式都保留了大值，区别在于小值是否还能彼此区分。FP8 也会舍入；它的优势是对这类数值分布更合适的刻度分配，而不是消除所有误差。

SmoothQuant 通过调整通道幅度，让数据适应 INT8 的均匀刻度；FP8 则选择非等距刻度。两者是不同的处理方式，实际选择仍需比较模型精度与执行速度。

3. 表示范围、精度与计算收益

FP8 将 8 位分配给符号、指数和尾数。指数决定数值的数量级，尾数决定同一数量级内的精细程度。

格式	符号位	指数位	尾数位	最大有限正数
E4M3FN	1	4	3	448
E5M2	1	5	2	57344

E5M2 范围更大，E4M3FN 在同一数量级内刻度更细。后续使用 E4M3FN，矩阵计算图中简写为 E4M3。范围与精度仍需取舍：大值附近的粗刻度会带来舍入误差，过小的值也可能下溢为零。FP8 格式

FP8 每个数占 1 字节，是 FP16/BF16 的一半；H100 的 Tensor Core 支持 FP8 矩阵乘法，理论吞吐量高于 FP16。**数值格式决定误差能否接受，硬件与内核决定低位宽能否转化为实际收益。** Hopper 架构

二、FP8 的量化过程与尺度选择

1. 缩放、舍入与还原

FP8 仍只有有限的范围和精度。设正数 s 为一组数据共享的量化尺度，先除以它，将数值限制到格式范围，再舍入到最近的 FP8 值；还原时乘回 s：

$$f = \mathrm{cast}_{\mathrm{E4M3FN}}(\mathrm{clip}(x/s, -448, 448)), \qquad \hat{x} = sf.$$

其中， clip 将超出范围的值截到边界， cast 将结果舍入为最近的 FP8 可表示值。 f 是 FP8 浮点值，不是整数编码。**乘回尺度能恢复数值大小，但不能消除截断与舍入产生的误差。**

2. 静态定标与动态定标

定标就是确定量化尺度。 对 E4M3FN，一种直接的做法是将这一组数的最大绝对值除以 448：

$$s = \max\left(\frac{\max|x|}{448}, \epsilon\right).$$

$\epsilon > 0$ 用于防止全零数据产生零尺度。对于激活，静态和动态都可以用这个公式，区别是**用哪批数据计算，以及算出的尺度是否复用** 。

	静态定标	动态定标
数据来源	提前准备的校准输入所产生的激活	当前真实输入所产生的激活
计算时机	部署前运行模型，统计激活范围并确定尺度	推理时，当前激活产生后再确定尺度
复用方式	后续输入复用固定尺度	当前这组激活使用，新的一组重新计算

例如，提前用校准输入运行模型，选定激活范围为 $[-224, 224]$，静态尺度就是 $224/448 = 0.5$ 。如果某次真实输入产生的激活最大绝对值为 448，且包含数值 448，两种方式的处理如下：

- **静态：** 仍用 0.5，缩放后得到 $448/0.5 = 896$，超出 FP8 范围，截断到 448 后只能还原为 224。
- **动态：** 根据当前范围算出尺度 $448/448 = 1$，这个激活可以还原为 448。

静态省去每次统计范围的开销，但固定尺度可能不适合新输入；动态能适应当前范围，但增加运行时统计。**两种方式都要量化当前激活，“静态”指尺度固定，不是激活不变。** 尺度与量化

权重则不同：它已经保存在模型中，推理期间通常不变，可以直接读取权重、统计范围并计算尺度，供后续推理复用，无需先用输入运行模型。

3. 量化粒度与尺度共享

量化粒度决定哪些数共享同一个尺度。整个张量共用一个尺度称为 **per-tensor**；每个 token 的激活向量各用一个尺度称为 **per-token**；权重也可以按输出通道分别设置尺度。更细的分组可减小大值对其他组的影响，但需要保存和处理更多尺度，实际误差与执行开销仍需评估。

三、FP8 线性层计算与尺度恢复

1. FP8 点积与尺度恢复

线性层计算 $Y = XW^\top$ 。输入 $X \in \mathbb{R}^{M\times K}$ 按 token 排列，权重 $W \in \mathbb{R}^{N\times K}$ 按输出通道排列； M 是 token 数， K 是输入通道数， N 是输出通道数。矩阵乘法也常称为 GEMM。

量化后得到 FP8 数据 F_X, F_W 。激活每行共享尺度 $s_X[m]$ ，权重每行共享尺度 $s_W[n]$ 。把还原值代入一个输出元素的点积：

$$\begin{aligned}\widehat{Y}_{mn} &= \sum_k \big(s_X[m]F_X[m,k]\big)\big(s_W[n]F_W[n,k]\big) \\ &= s_X[m]s_W[n]\sum_k F_X[m,k]F_W[n,k].\end{aligned}$$

两个尺度都不随求和位置 k 改变，所以可以**直接用 FP8 数据做矩阵乘法，再将每个输出乘以对应的两侧尺度** ，无需先将完整的输入和权重矩阵还原为高精度。

将激活尺度写成 $M \times 1$ 的列向量 s_X ，权重尺度写成 $N \times 1$ 的列向量 s_W ，矩阵形式为：

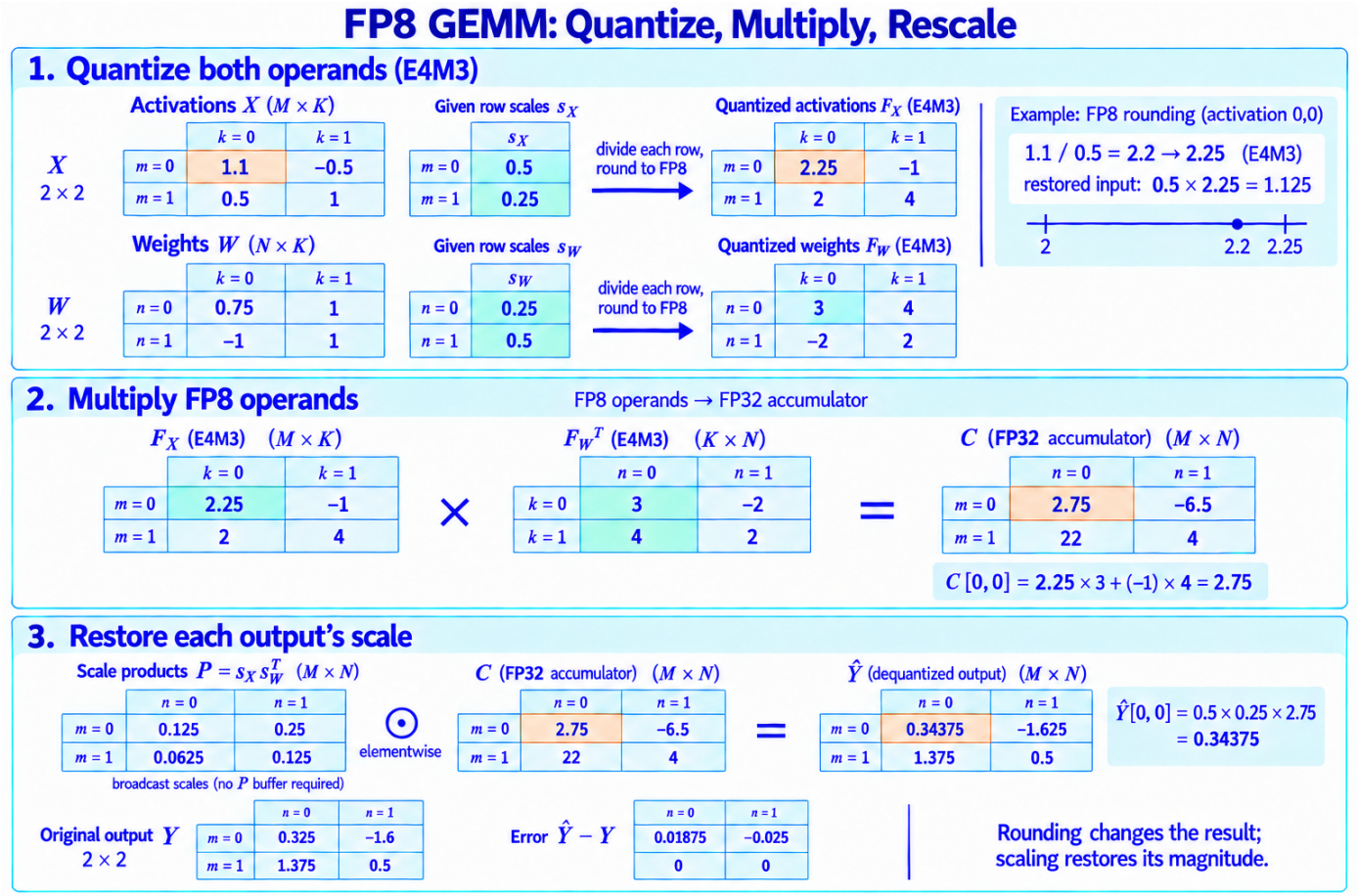
$$C = F_X F_W^\top, \qquad \widehat{Y} = (s_X s_W^\top) \odot C.$$

$\odot$ 表示对应位置相乘。 $s_X s_W^\top$ 给出每个输出对应的尺度乘积；实际内核可以直接读取两侧尺度，不必存储该外积矩阵。

操作数用 FP8，不代表累积结果也用 FP8。 一个点积要将许多乘积相加，若部分和也反复舍入到 FP8，较小的加数可能因舍入而丢失，部分和还可能超出表示范围。因此，下文的 Hopper CUTLASS FP8 路径使用 FP32 累积，最后再恢复尺度并转换为输出类型。

2. 两个 token 的完整算例

图中使用两个 token、两个输入通道和两个输出通道。激活行尺度为 $[0.5, 0.25]^\top$ ，权重行尺度为 $[0.25, 0.5]^\top$ ，按“量化 → 点积 → 恢复尺度”计算：



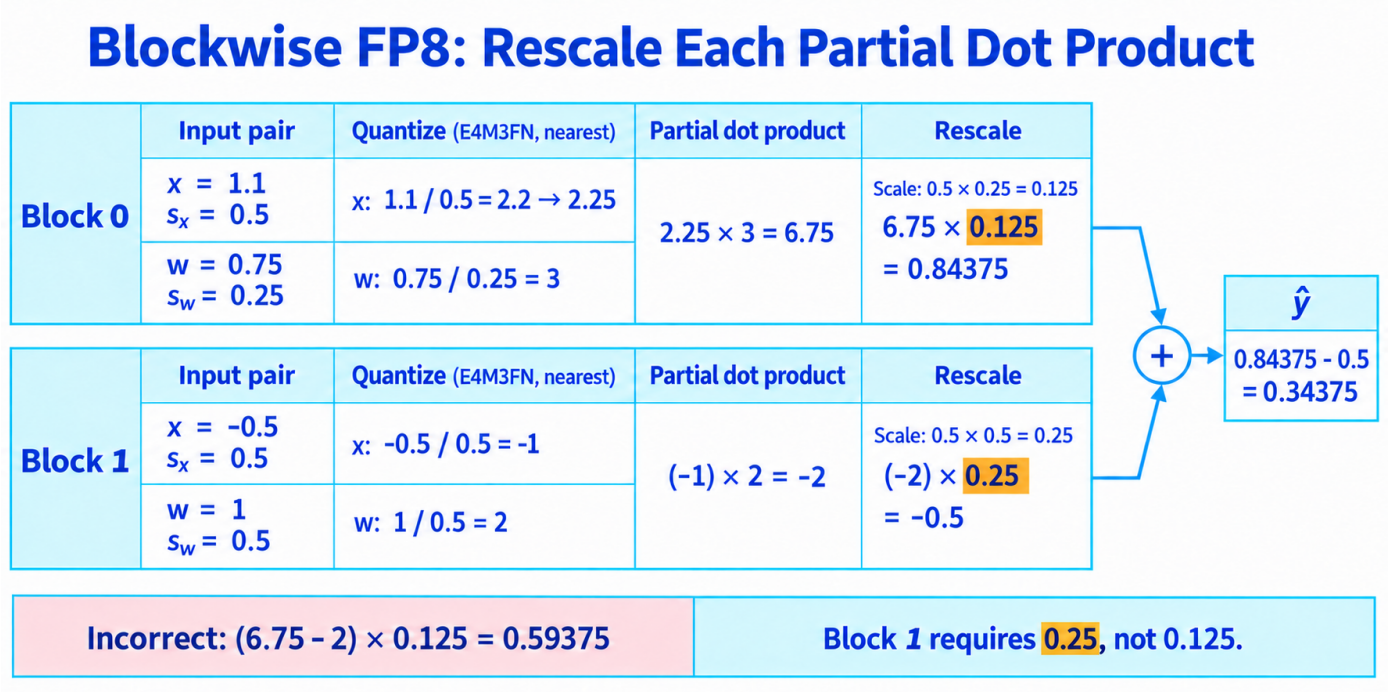
图中第一项激活的舍入误差，经两个输出通道的权重分别加权，形成最终的输出误差。尺度恢复不会将这部分误差抵消。

3. 分块量化中的尺度恢复

前面的按行量化中，一个 token 的整行激活共享一个尺度，一行权重也共享一个尺度。如果同一行内不同区域的数值范围差异较大，可以沿输入通道切成若干块，每块单独设置尺度，减少大值对其他块的影响。计算点积时，激活与权重按对应的通道位置配对。

按行量化时，整个点积共用同一个尺度乘积，所以可以先完成点积，再统一恢复尺度。**分块后，各块的尺度乘积可能不同，需要先恢复各块的部分点积，再将结果相加。** 这里的"部分点积"，就是只对一个块内的通道做乘法求和。

下图将激活行 $x = [1.1, -0.5]$ 与权重行 $w = [0.75, 1]$ 的两个通道分别作为一块。两块的激活尺度均为 0.5，权重尺度分别为 0.25 和 0.5：



图中两块分别乘回 0.125 和 0.25。若先把 FP8 部分点积相加，再统一乘 0.125，第二块的贡献就会被错误地缩小一半。

一个块包含多个通道时，计算过程相同。用 g 表示块号， m 表示 token， n 表示输出通道； $s_X[m, g]$ 和 $s_W[n, g]$ 分别是对应块的激活尺度与权重尺度。先计算块内的 FP8 部分点积 $C_{mn}^{(g)}$ ，再恢复尺度并跨块求和：

$$C_{mn}^{(g)} = \sum_{k \in g} F_X[m, k] F_W[n, k],$$

$$\widehat{Y}_{mn} = \sum_{g} s_X[m, g] s_W[n, g] C_{mn}^{(g)}.$$

块内共享尺度，尺度可以从块内求和中提取；跨块时则保留各自的尺度乘积。这些操作可以在同一个矩阵计算内核中完成。

四、vLLM 中的 FP8 加载与执行

FP8 W8A8 表示权重和激活都采用 8 位浮点格式。以 **vLLM v0.25.1** 的 `compressed-tensors` 配置为例，激活按 token、权重按输出通道共享尺度，在 Hopper GPU 上通过 CUTLASS 矩阵计算内核执行：

1. **加载权重与尺度。** 读取 FP8 权重和权重尺度，将权重从 $[N, K]$ 转为矩阵乘法需要的 $[K, N]$ 视图，供后续请求复用。加载处理
2. **量化当前激活。** 接收浮点输入时，根据当前各 token 的输入范围生成尺度，并得到 FP8 激活。静态模式则使用已有尺度，但仍需量化当前输入。执行入口
3. **计算并恢复输出。** 将 FP8 激活、FP8 权重及两侧尺度一起交给 `cutlass_scaled_mm()` 。所选内核使用 E4M3 操作数、FP32 累积，再乘尺度、有偏置时加偏置，最后转为输出类型。内核调用、累积类型、E4M3 配置

尺度恢复在矩阵内核的输出阶段完成，这一阶段称为 **epilogue** 。无需先将完整的累积矩阵 C 写回显存，再启动一次缩放。输出处理

使用分块尺度时，vLLM 选择对应的 block-scaled 内核；这条 CUTLASS 路径要求激活每个 token 按 128 个输入通道分组，按组处理尺度。分组路径

实际收益要比较完整线性层的耗时，包括激活范围统计、量化、矩阵乘法和输出处理，而不只比较 FP8 矩阵乘法本身。